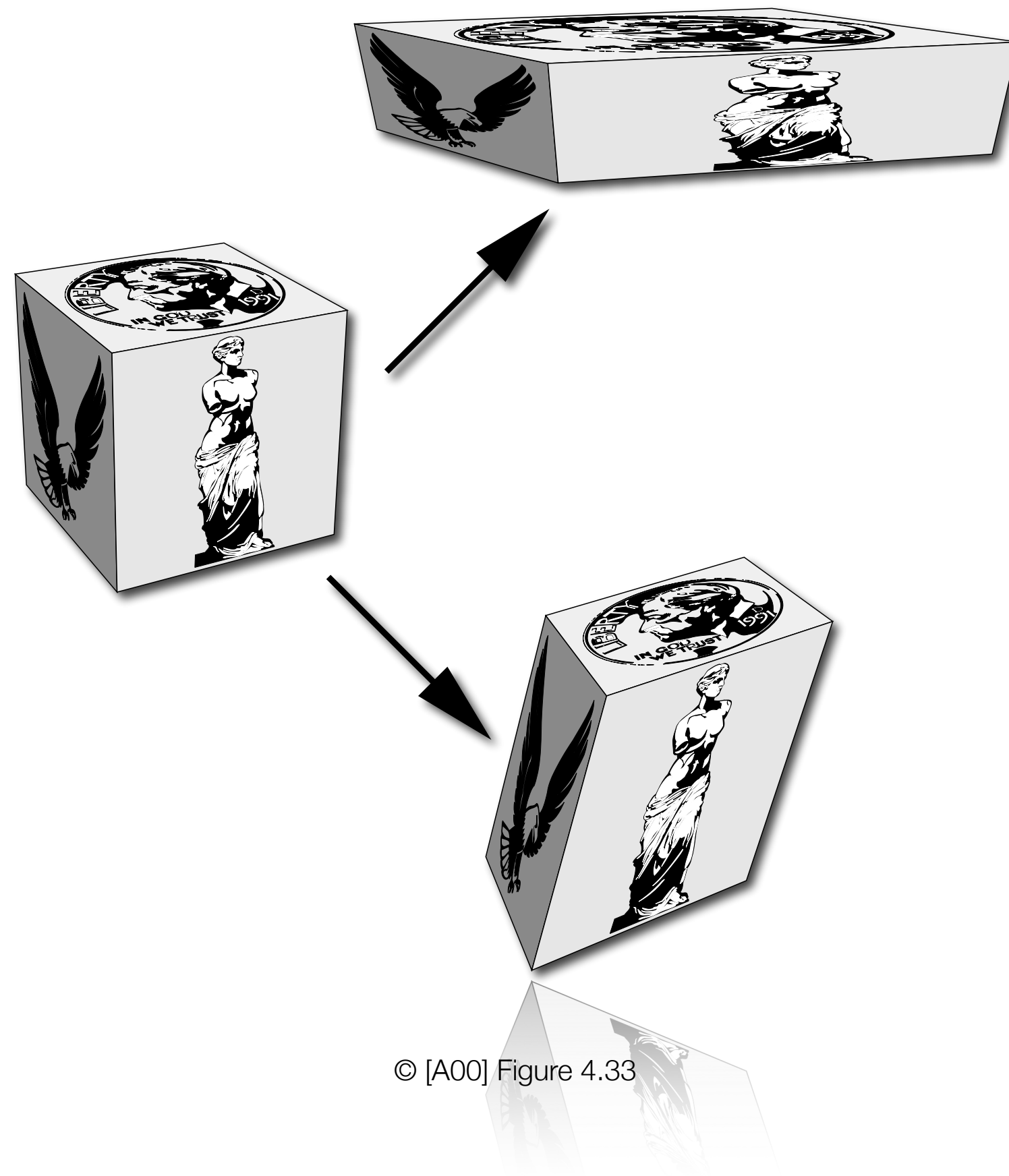


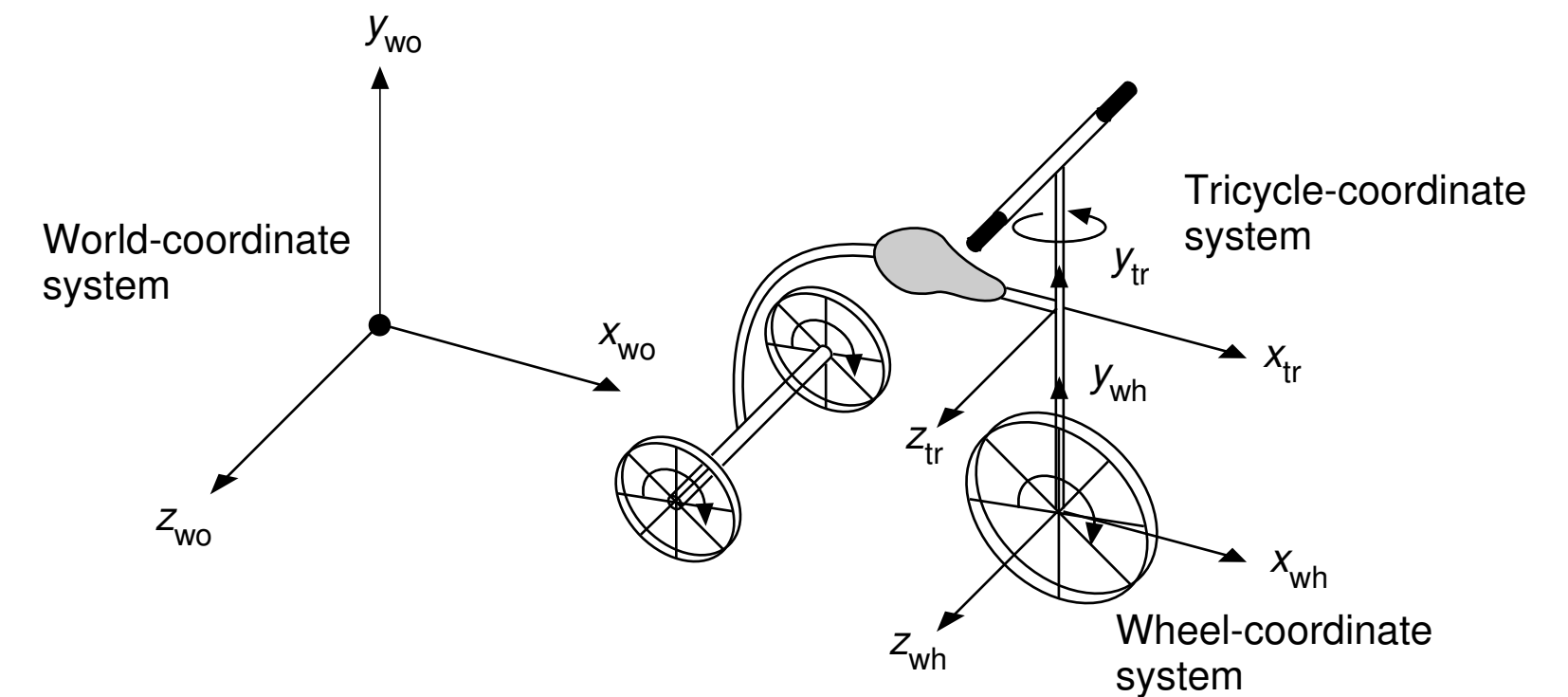
Transformations



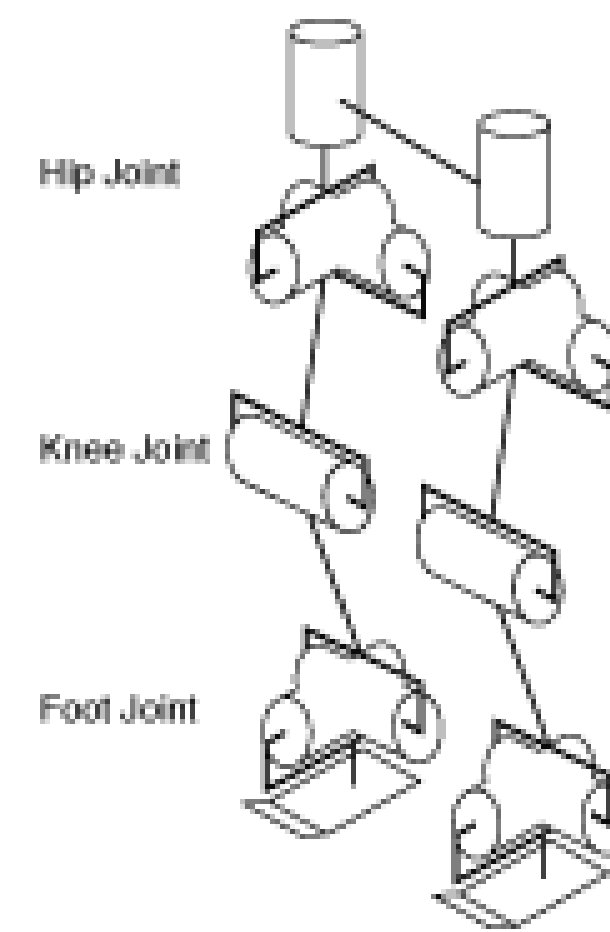
© [A00] Figure 4.33

Overview of Transformations

- Transformations correspond to a change of the coordinate system frame
 - ▶ transformations applied to the entire 3D scene, and all its objects, correspond to a global change in the coordinate system reference frame
- Transformations can be applied separately to individual objects or even locally to separate parts
 - ▶ model and animate complex objects such as robots through transforming their parts
- Transformations are efficiently represented and implemented by matrix multiplications
 - ▶ use of homogeneous coordinates



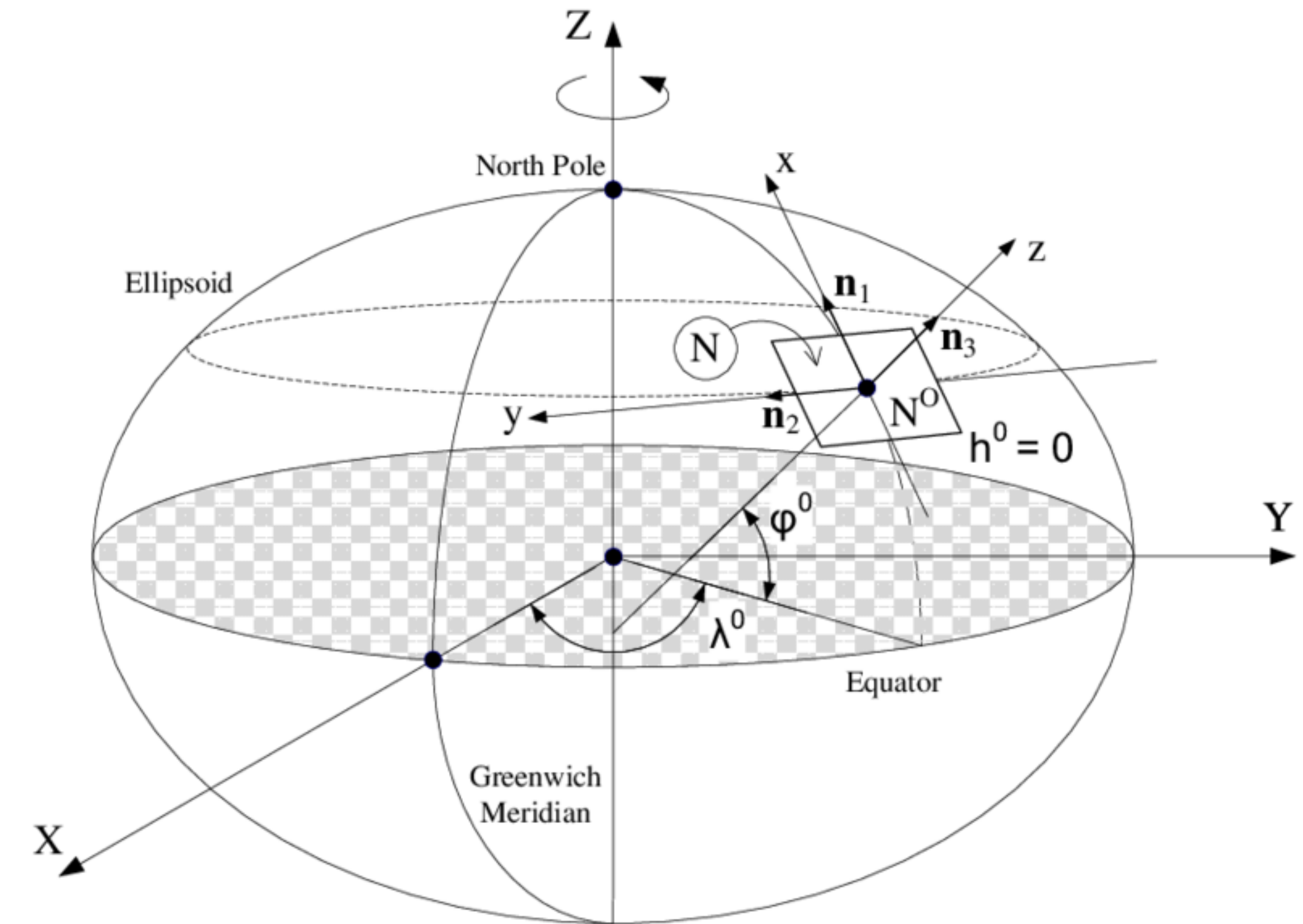
© [AW94] Figure 5.28



Coordinate Systems and Frames

Frame of Reference

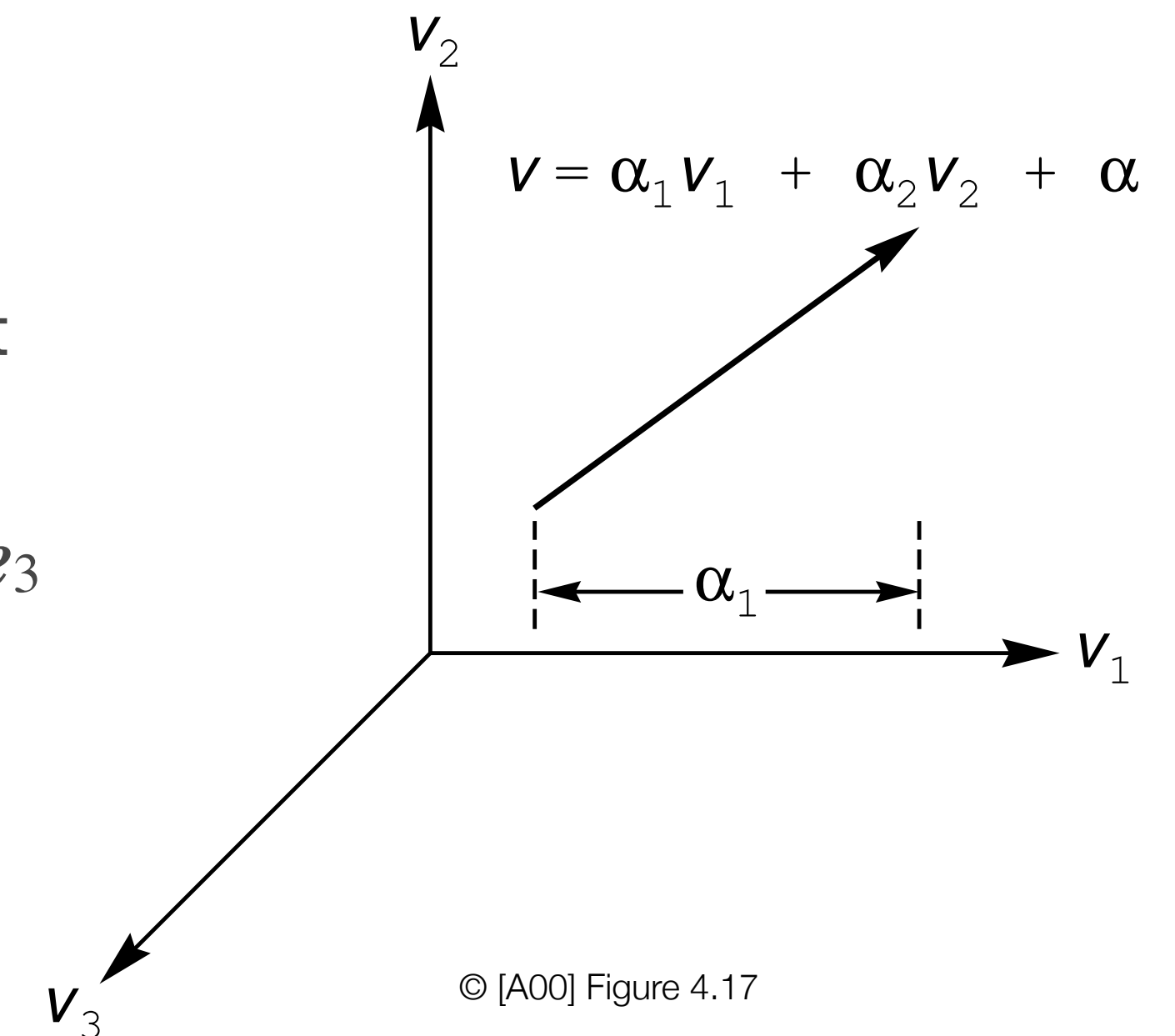
- A geometric object is defined by coordinates in a given coordinate system, its local frame of reference
- The frame of reference is flexible and can be changed
 - ▶ change of coordinate system
 - ▶ described by a transformation of coordinates



© Sandino, Luis. Modeling and control techniques of autonomous helicopters for landing on moving platforms. PhD Thesis, Universidad de Sevilla, 2016.

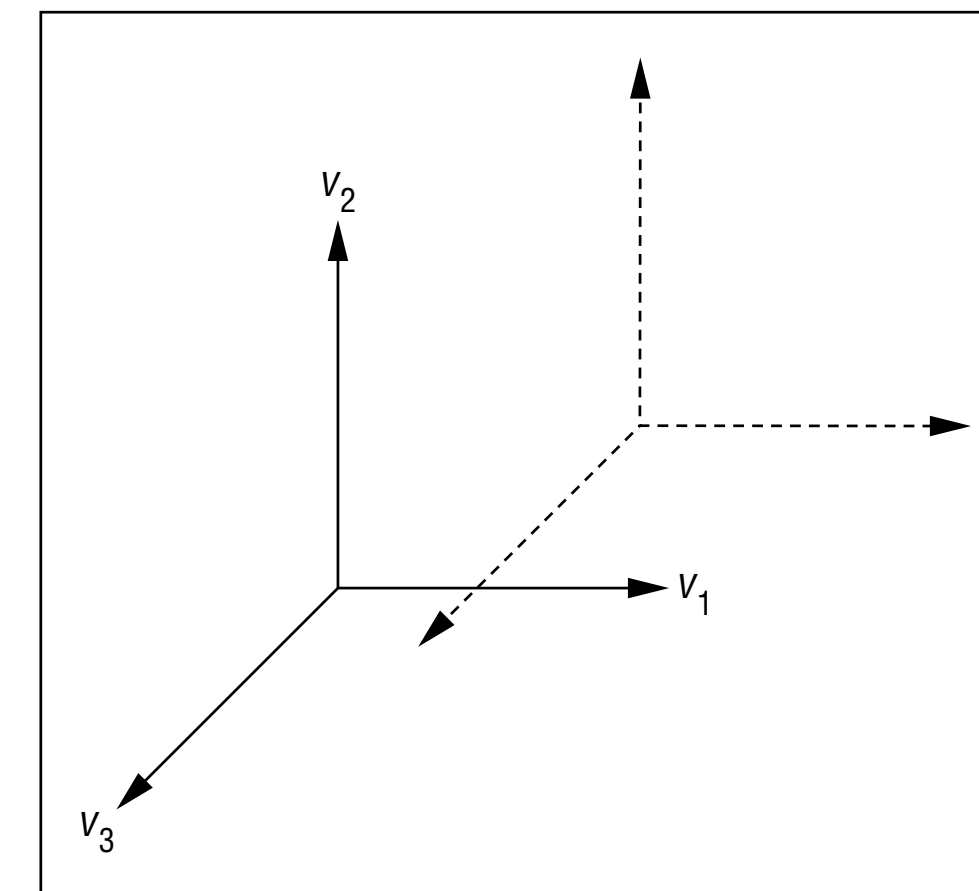
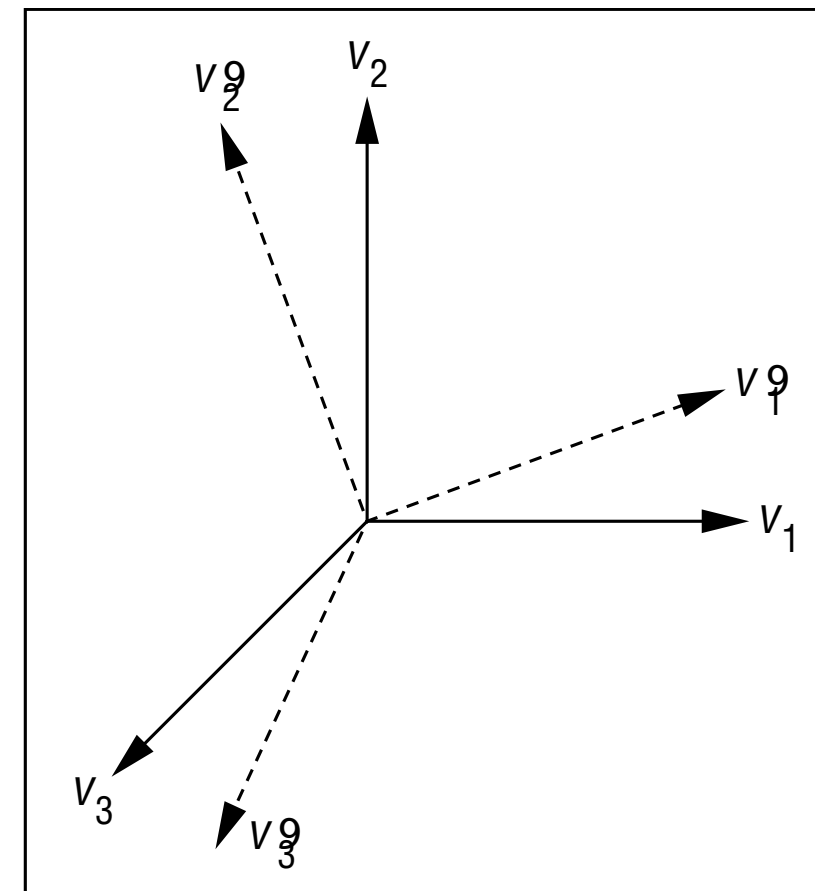
Coordinate System Frame

- To represent n -dimensional points as well as vectors we need an affine vector space
 - ▶ fix particular reference point – the origin as $P_0 = [0, 0, 0]$
- Use orthonormal unit-length basis vectors
 - ▶ Cartesian 3D basis $e_1 = [1, 0, 0]^T$, $e_2 = [0, 1, 0]^T$, $e_3 = [0, 0, 1]^T$
- Thus a frame is defined by its origin P_0 and the unit-length basis vectors $e_{1...n}$ defining the orientation of the coordinate system axis
 - ▶ n -dimensional points are represented as coordinate n -tuples with respect to a point of origin and a set of n basis vectors
- Each 3D point $P = [\alpha_1, \alpha_2, \alpha_3]^T$ can be written as $P = P_0 + \alpha_1 e_1 + \alpha_2 e_2 + \alpha_3 e_3$
 - ▶ representation in three-dimensional Euclidean
 - ▶ similar for 3D vectors $v = v_1 e_1 + v_2 e_2 + v_3 e_3$
- Origin and basis determine the coordinate system's frame



© [A00] Figure 4.17

Change of Coordinate System



© [A00] Figures 4.19 and 4.20

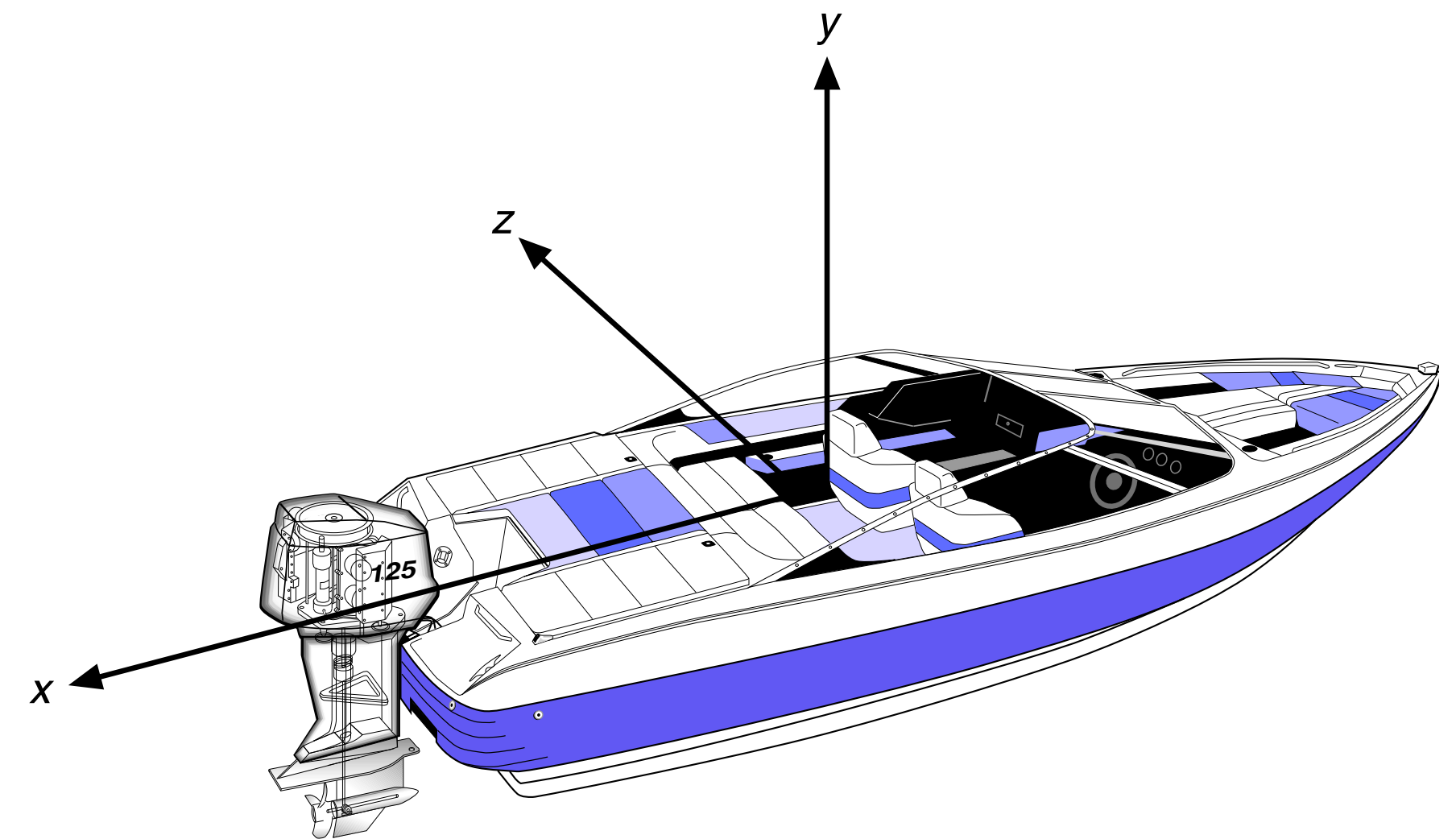
- Change of basis vectors or change of origin causes change of the coordinate system reference frame
 - ▶ change of basis vectors $\Rightarrow$ **rotation**
 - ▶ change of origin $\Rightarrow$ **translation**

Frames in Graphics

- In a computer graphics system a number of different frames and representations are typically used:
 1. object or model coordinates
 2. world coordinates
 3. camera coordinates
 4. clip coordinates
 5. normalized device coordinates
 6. window or viewport coordinates

Model Coordinates

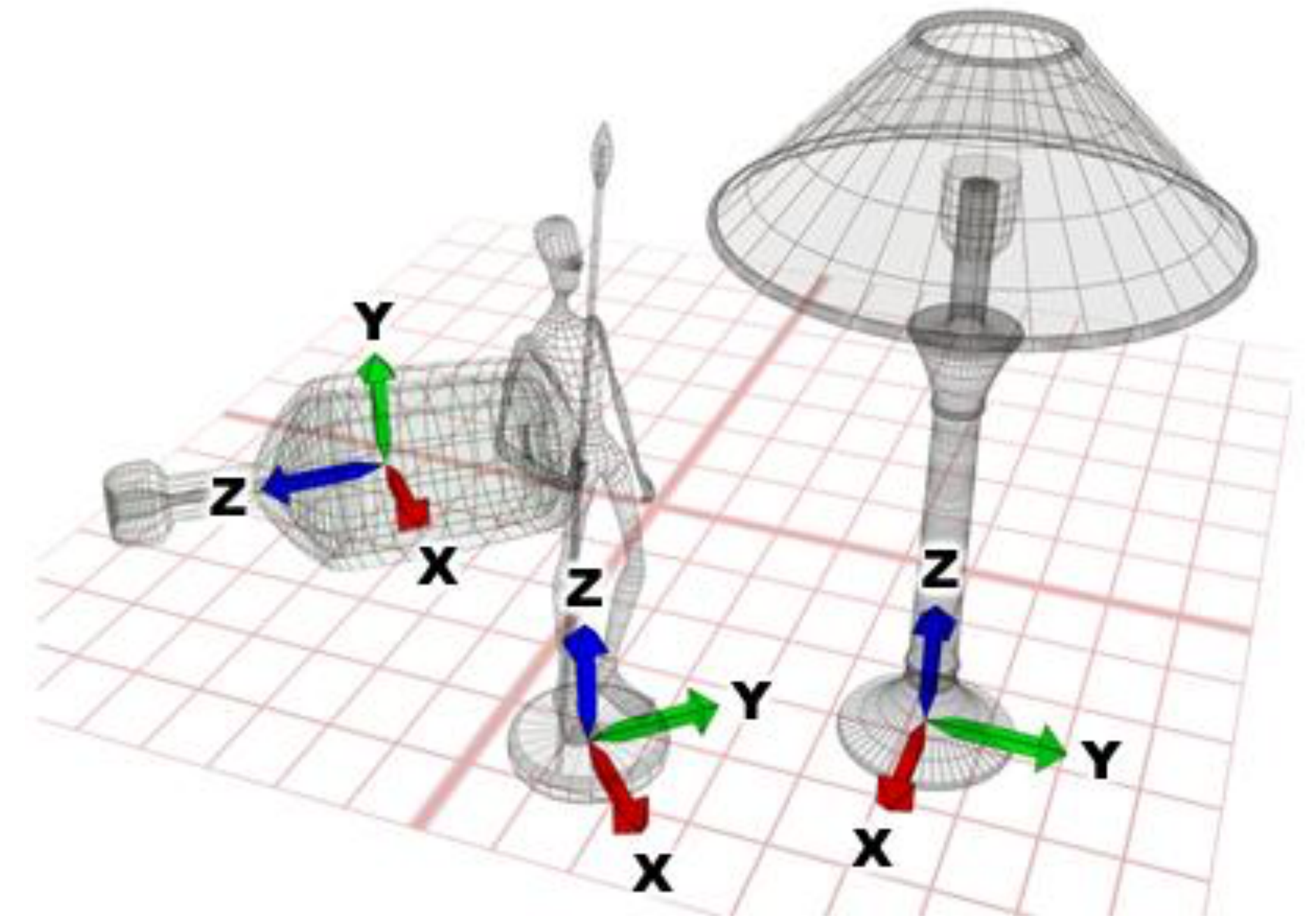
- Geometry is typically defined object-centric in a local coordinate system
 - ▶ convenient scale, orientation and origin for designing and representing a single object
- Multiple objects populating the entire 3D world must be scaled and oriented appropriately
 - ▶ in their local coordinate reference frames by local transformations



© [AW94] Figure 6.01

World Coordinates

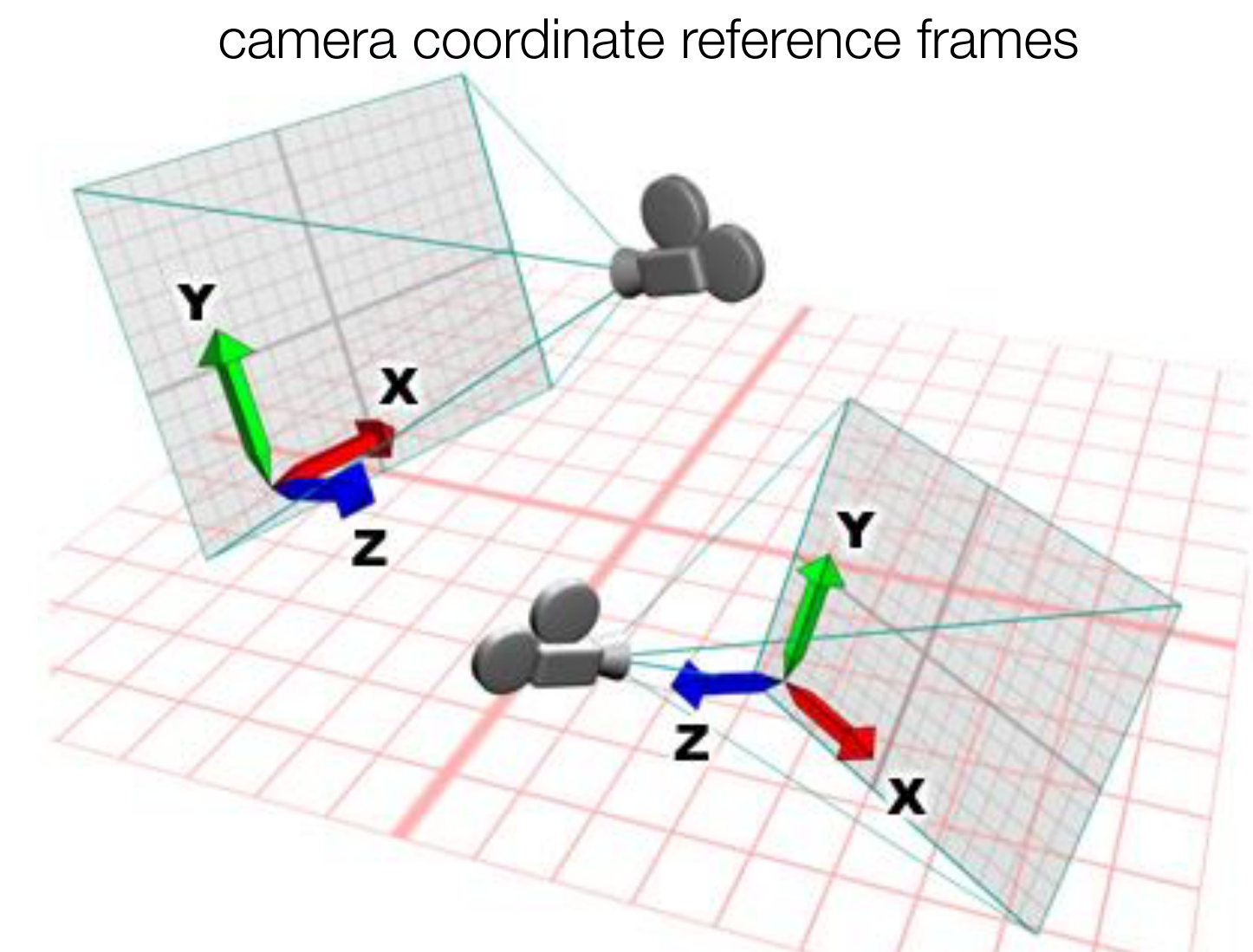
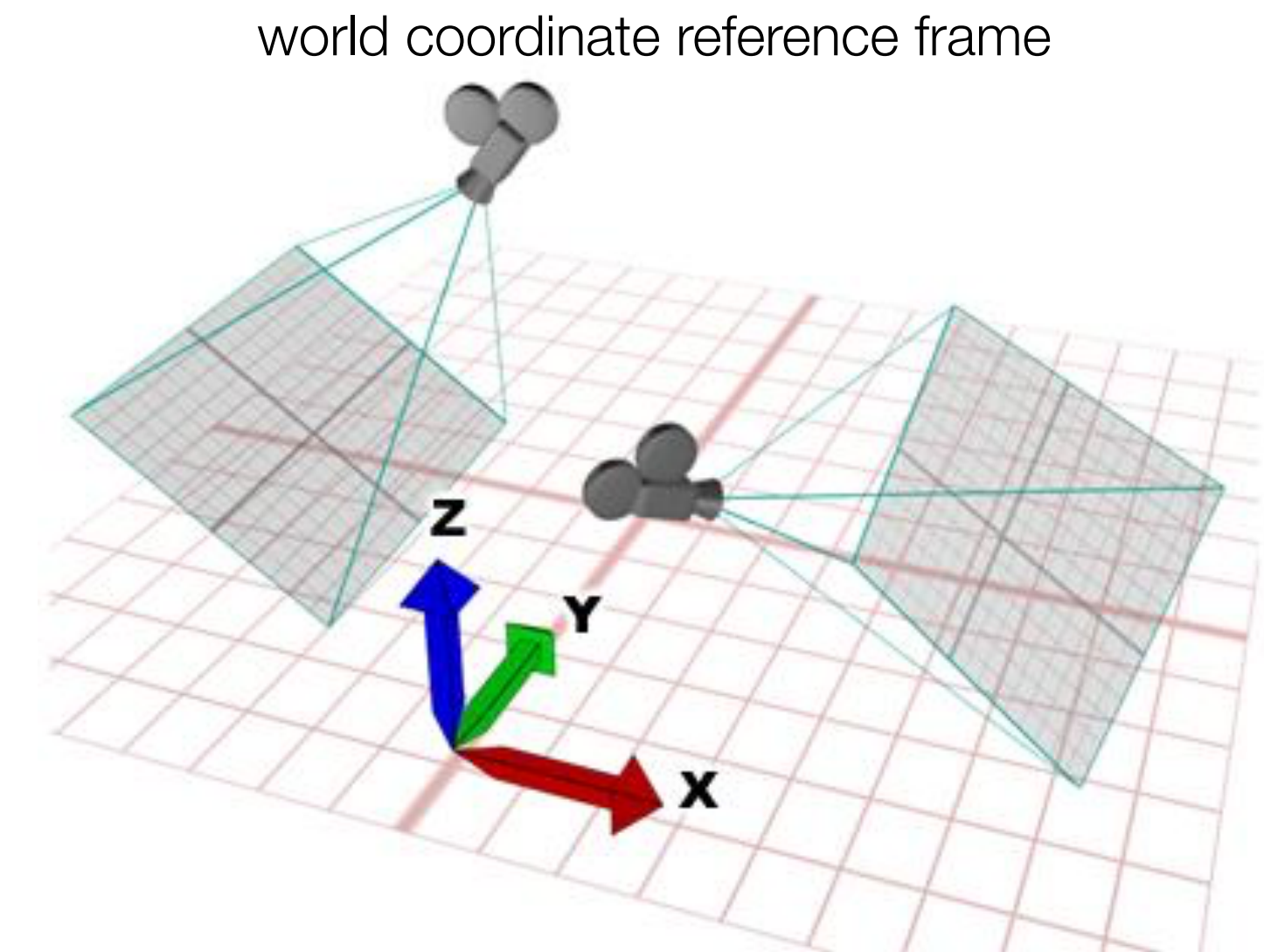
- Entire scene defining the **world** to be displayed consists of many objects
 - ▶ environment and all objects need to be represented in the same global coordinate system
- World coordinate (WC) frame defined by user-chosen coordinate axis and origin
 - ▶ e.g. canonical geographical coordinates
 - ▶ default working coordinate system in OpenGL



Source: http://www.3dmax-tutorials.com/idh_tb_coord_sys.html

Camera Coordinates

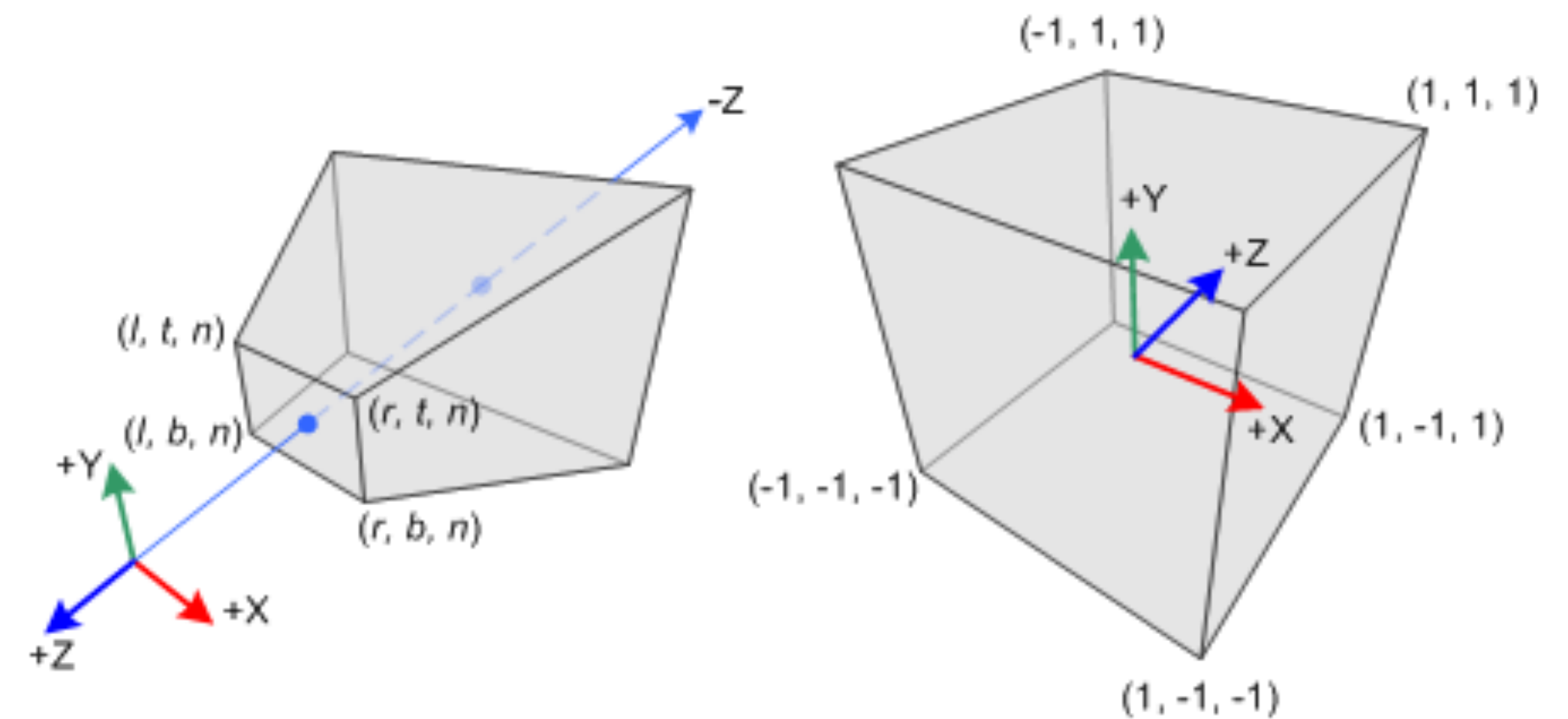
- Default frame of reference for graphics system is the camera
 - ▶ displayed image defined by what camera sees
 - ▶ camera lens at origin, and one axis aligned with the camera's viewing direction
- Model-view transformation
 - ▶ change of representation from model to world and then the camera reference frame



Source: http://www.3dmax-tutorials.com/idh_tb_coord_sys.html

Clip Coordinates

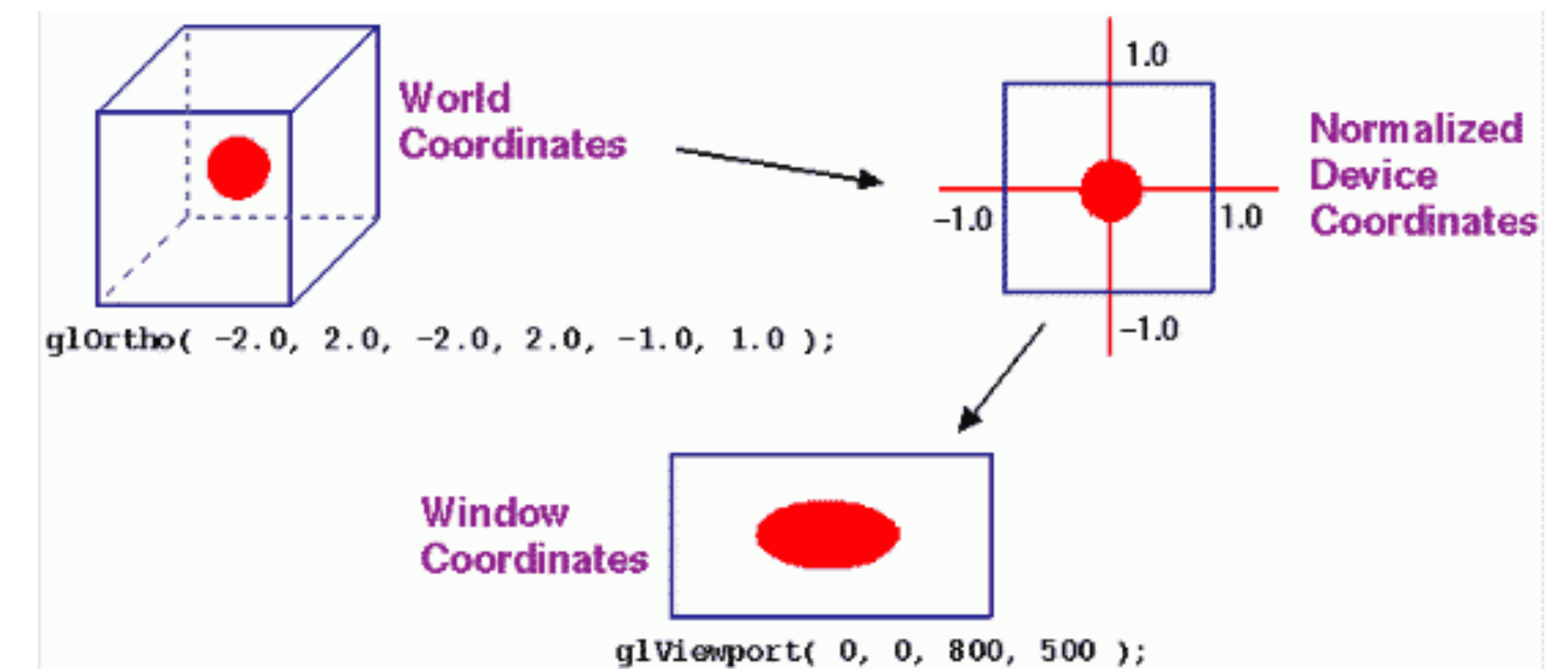
- View of camera is limited by field-of-view, minimal and maximal viewing distance, called **view frustum**
 - ▶ geometry must be clipped to the visible view frustum
- Visibility clipping is performed after the transformation into a **canonical viewing volume**
 - ▶ unit sized pyramid, or in general
 - ▶ unit sized cube centered at origin



Source: http://www.songho.ca/opengl/gl_projectionmatrix.html

NDC and Window Coordinates

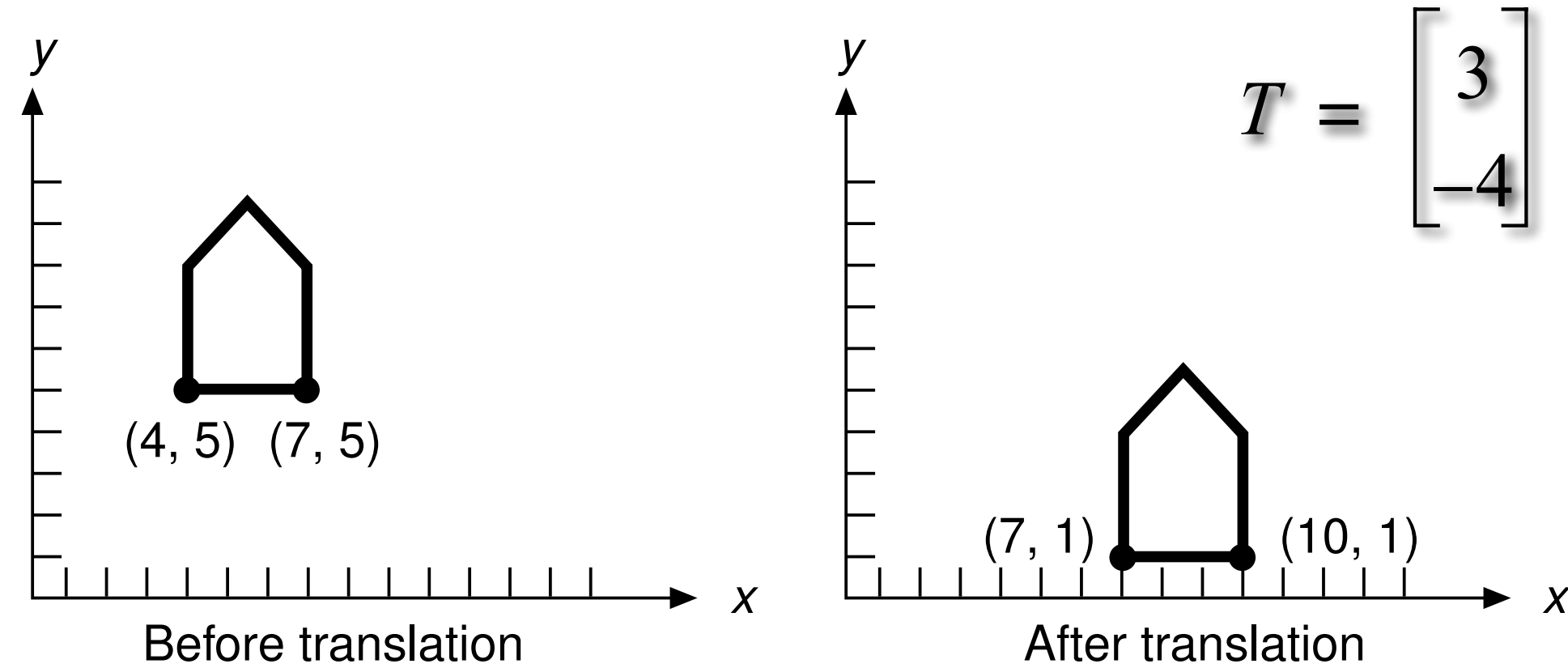
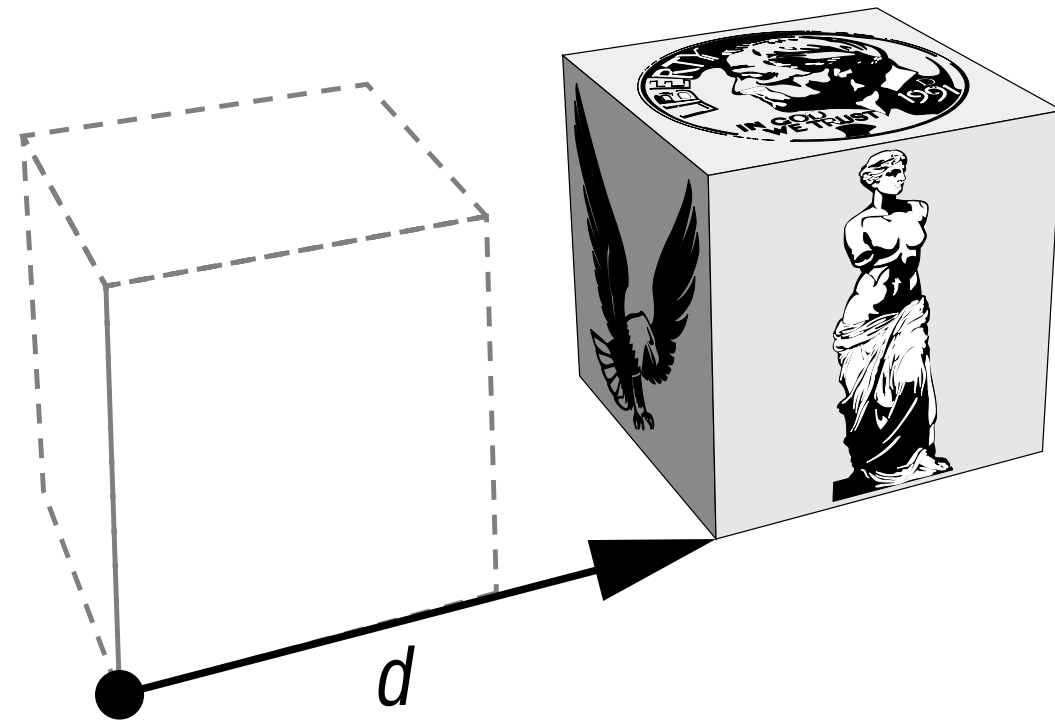
- Perspective division of homogenous clip coordinates transforms the object representation into the Normalized Device Coordinates (NDC)
- Viewport transformation converts NDC to window coordinates
 - ▶ corresponding to pixel locations on screen



Source: <https://stackoverflow.com/questions/18423973/convert-to-pixel-coordinates-in-jogl-opengl>

2D Transformations

Translation

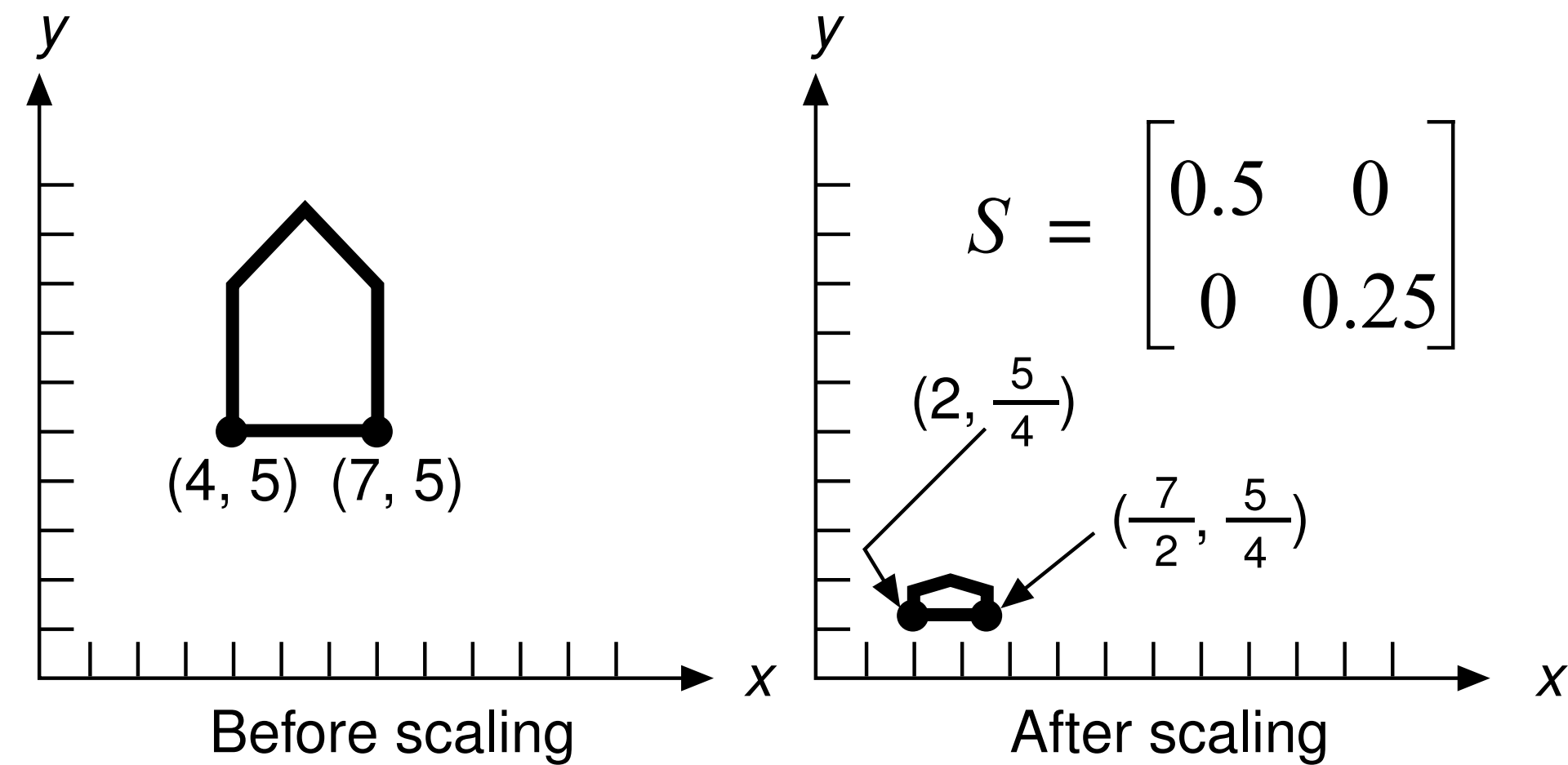
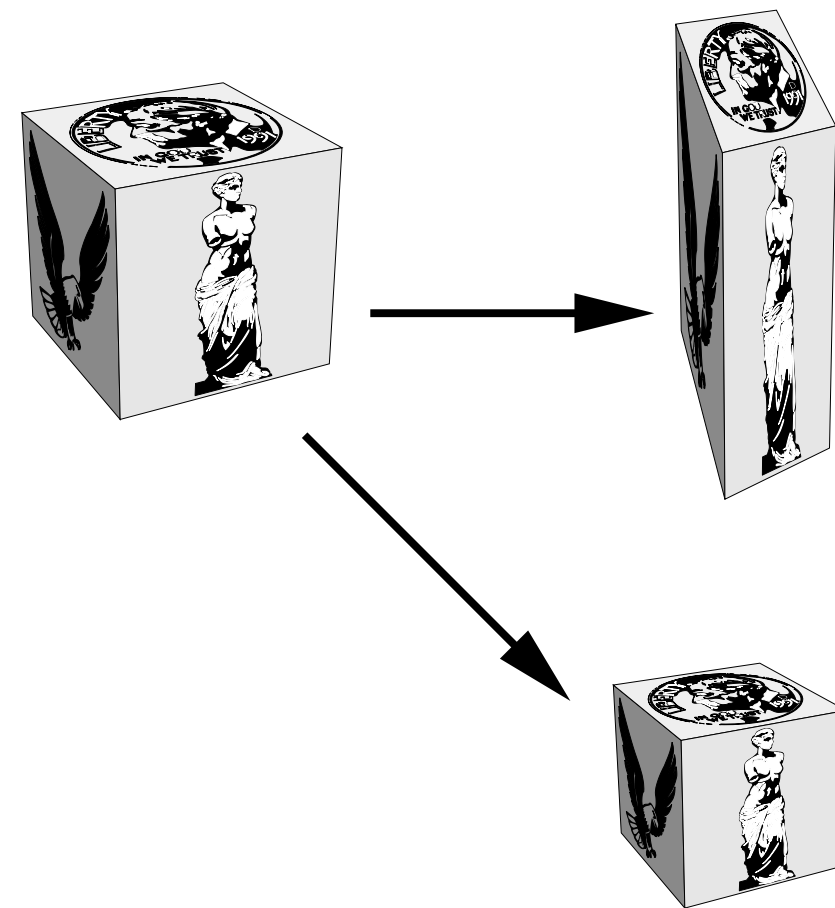


© [A00] Figure 4.29 and [AW94] Figure 5.03

- Translation of points achieved by addition of displacement terms d_x and d_y by component-wise vector addition
 - does not make reference to a certain frame or representation

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} d_x \\ d_y \end{bmatrix} \text{ or } P' = P + T$$

Scaling

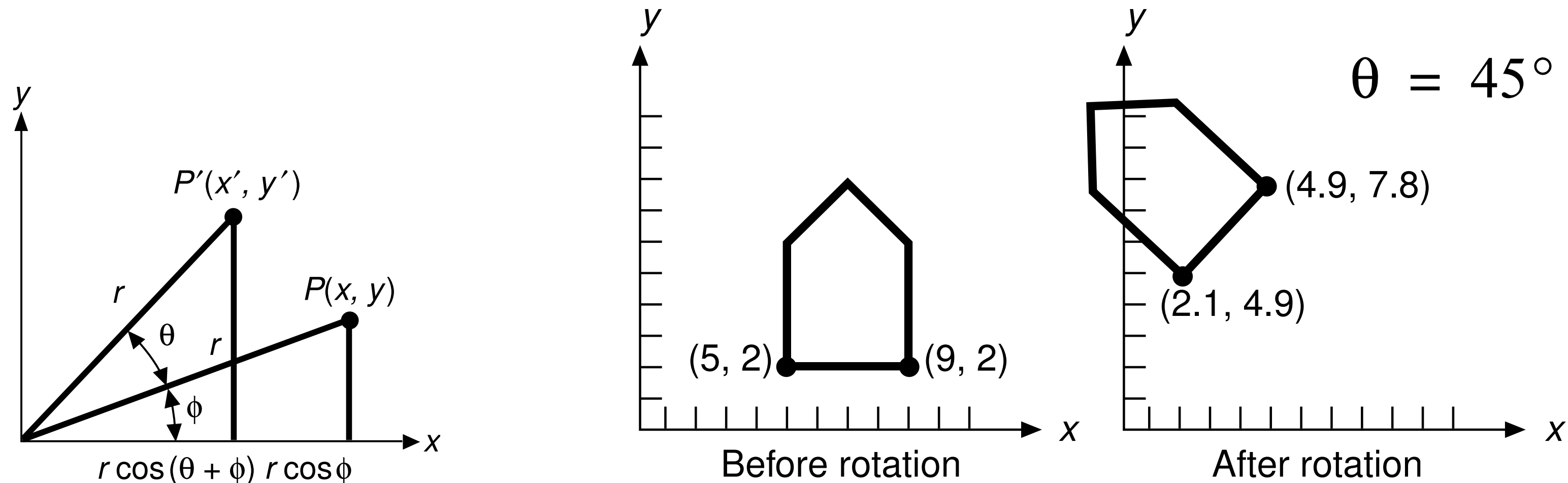


© [A00] Figure 4.34 and [AW94] Figure 5.04

- Points are scaled by multiplication of their origin-vectors with factors s_x and s_y by
 - ▶ $x' = s_x \cdot x$ and $y' = s_y \cdot y$
 - ▶ but changes location of objects in space!

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \text{ or } P' = S \cdot P$$

Rotation



© [AW94] Figures 5.05 and 5.06

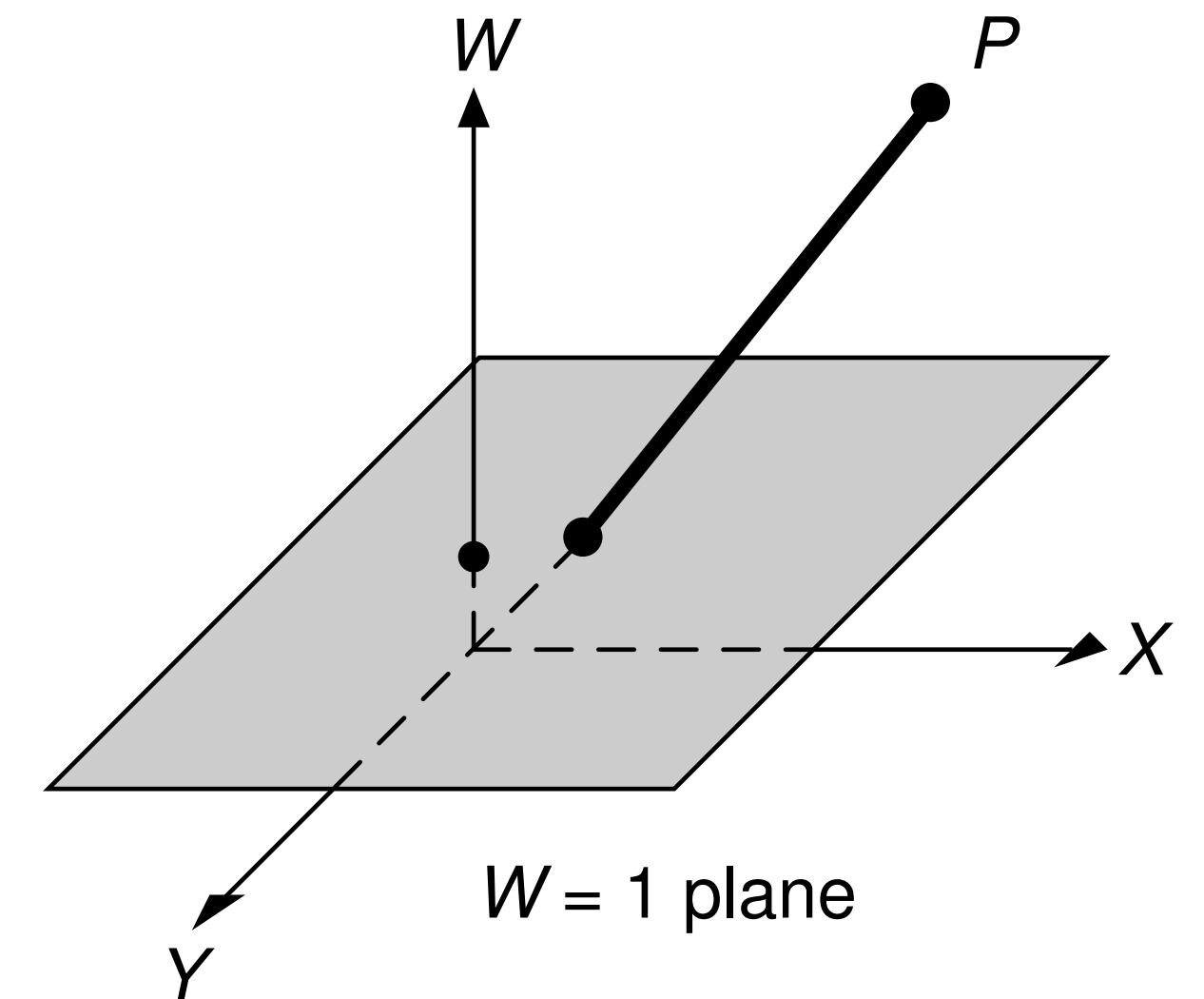
- Rotation is defined by rotation axis and rotation angle θ (counterclockwise)
 - ▶ $x' = x \cos\theta - y \sin\theta$ and $y' = x \sin\theta + y \cos\theta$
 - ▶ but changes location of objects in space!

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \text{ or } P' = R \cdot P$$

Homogeneous 2D Transforms

Homogeneous Coordinates

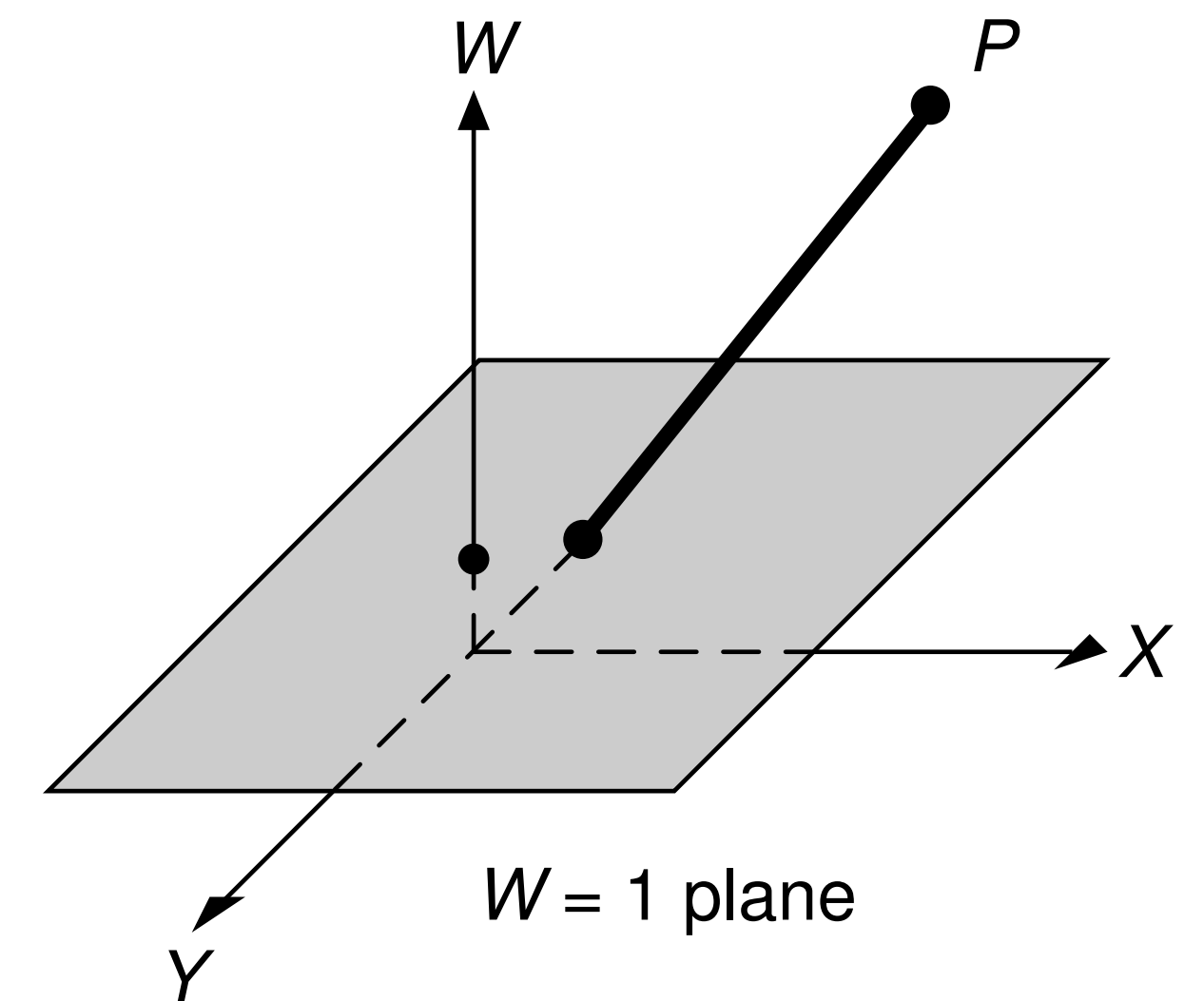
- The homogeneous representation of an n -dimensional space has $n+1$ coordinates
 - ▶ $[x, y]^T \rightarrow [X, Y, W]^T$
- Homogeneous points are said to be equal if and only if being **scalar multiples**
 - ▶ i.e. $[2, 3, 6]^T \equiv [4, 6, 12]^T$
- Any Cartesian point has an infinite number of homogeneous equivalents
 - ▶ defines **equivalence class** $[t \cdot X, t \cdot Y, t \cdot W]^T$ with $t \neq 0$
 - ▶ at least one coordinate must be nonzero
- Equivalence class defines line through the origin in homogeneous space



© [AW94] Figure 5.07

Homogeneous Coordinates

- Homogenization - division by W yields canonical point coordinate $[x, y, 1]^T$
 - ▶ with $[X, Y, W]^T \equiv [X/W, Y/W, 1]^T$
 - ▶ defines plane $W = 1$ of Cartesian representation
 - ▶ X/W and Y/W are Cartesian coordinates in n -space
 - ▶ points with $W = 0$ are considered at infinity
- Vectors are of type $[x, y, 0]^T$



© [AW94] Figure 5.07

Translation

- In homogeneous space we can express also translation as **matrix multiplication**

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & d_x \\ 0 & 1 & d_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \text{ with the **translation matrix** } T = \begin{bmatrix} 1 & 0 & d_x \\ 0 & 1 & d_y \\ 0 & 0 & 1 \end{bmatrix}$$

- Combination $P' = T^1 \cdot P$ and $P'' = T^2 \cdot P'$ yields expected **net translation** $d_{x,y} = d^1_{x,y} + d^2_{x,y}$

$$P'' = T^2 \cdot (T^1 \cdot P) = (T^2 \cdot T^1) \cdot P = \left(\begin{bmatrix} 1 & 0 & d^2_x \\ 0 & 1 & d^2_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & d^1_x \\ 0 & 1 & d^1_y \\ 0 & 0 & 1 \end{bmatrix} \right) \cdot P = \begin{bmatrix} 1 & 0 & d^1_x + d^2_x \\ 0 & 1 & d^1_y + d^2_y \\ 0 & 0 & 1 \end{bmatrix} \cdot P$$

Scaling

- Scaling is expressed as homogeneous **matrix multiplication** in the form of

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \text{ with the **scaling matrix** } S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Multiple scalings $P'' = T^2 \cdot (T^1 \cdot P)$ are **multiplicative** as expected $s_{x,y} = s^1_{x,y} \cdot s^2_{x,y}$

$$P'' = S^2 \cdot (S^1 \cdot P) = (S^2 \cdot S^1) \cdot P = \left(\begin{bmatrix} s_x^2 & 0 & 0 \\ 0 & s_y^2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} s_x^1 & 0 & 0 \\ 0 & s_y^1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \cdot P = \begin{bmatrix} s_x^1 \cdot s_x^2 & 0 & 0 \\ 0 & s_y^1 \cdot s_y^2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot P$$

Rotation

- Rotational **matrix multiplication** is

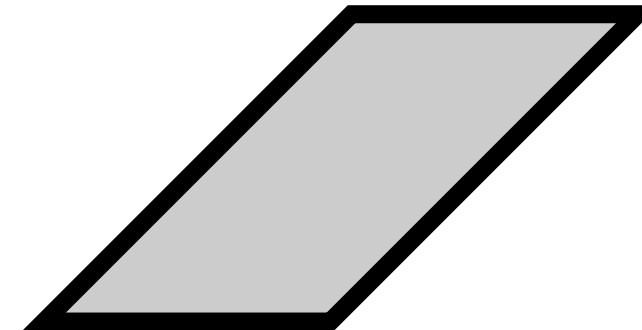
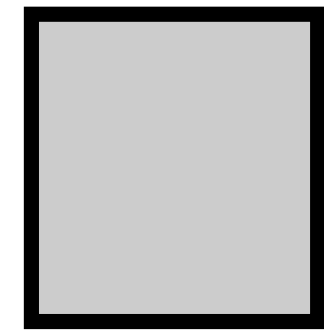
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \text{ with the **rotation matrix** } R = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Successive rotations yield expected result with additive rotation angles

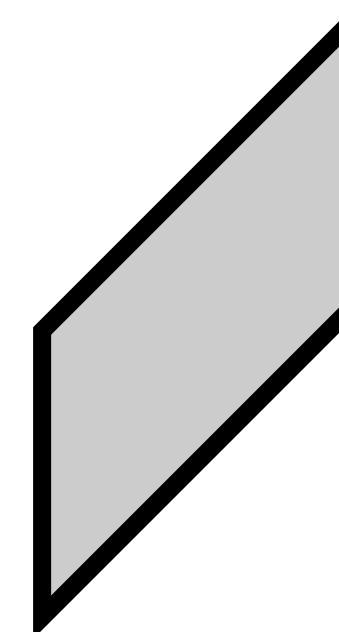
$$\begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 \\ \sin \theta_2 & \cos \theta_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & 0 \\ \sin \theta_1 & \cos \theta_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos \theta_1 \cdot \cos \theta_2 - \sin \theta_1 \cdot \sin \theta_2 & -\sin \theta_1 \cdot \cos \theta_2 - \cos \theta_1 \cdot \sin \theta_2 & 0 \\ \cos \theta_1 \cdot \sin \theta_2 + \sin \theta_1 \cdot \cos \theta_2 & -\sin \theta_1 \cdot \sin \theta_2 + \cos \theta_1 \cdot \cos \theta_2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos(\theta_1 + \theta_2) & -\sin(\theta_1 + \theta_2) & 0 \\ \sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Shear



The unit cube sheared in the x direction



The unit cube sheared in the y direction

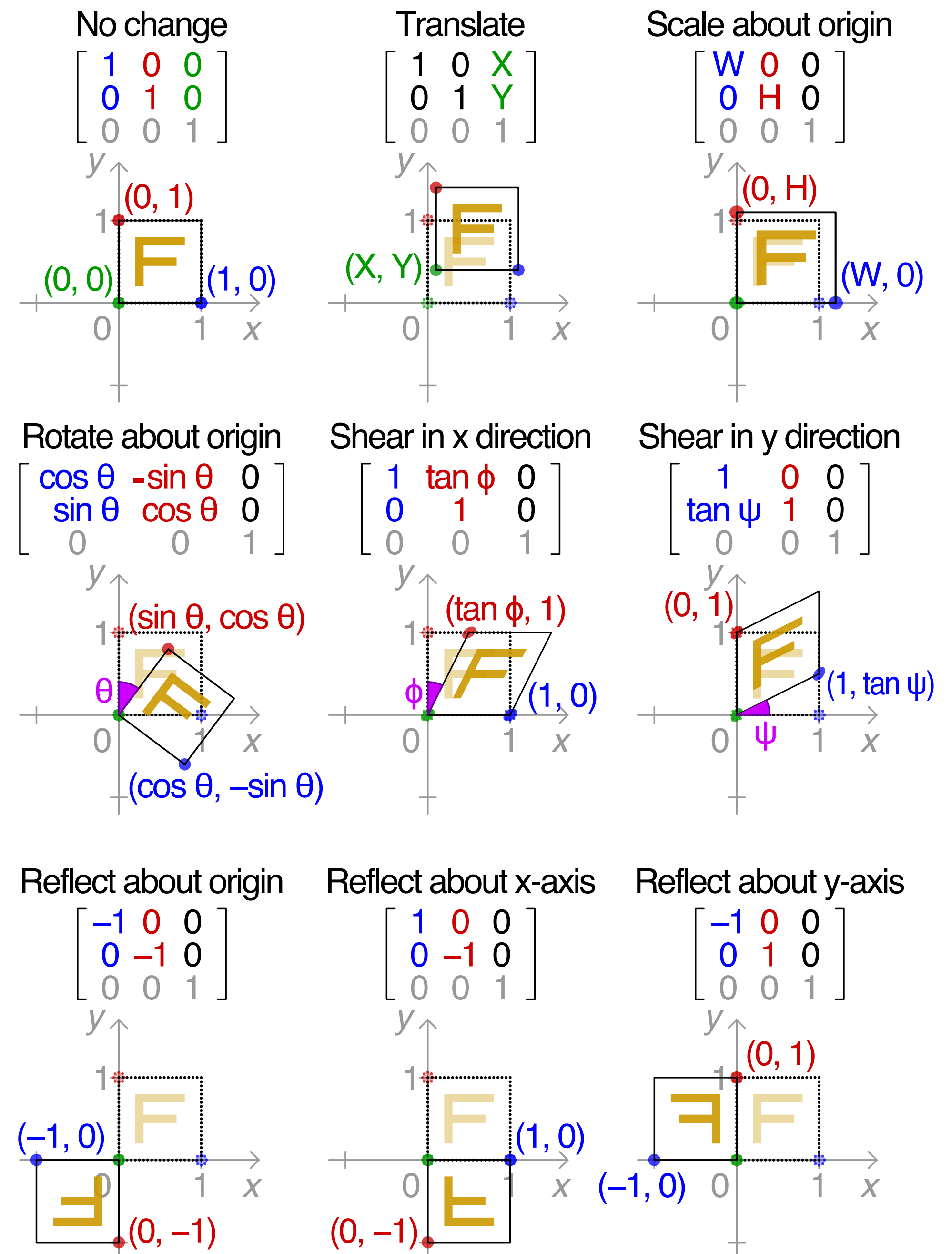
© [AW94] Figure 5.09

- Shear is performed along a hyper-plane of the coordinate system
 - ▶ x- or y-axis in 2D
 - ▶ $x' = x + a \cdot y$ and $y' = y + b \cdot x$

$$SH_x = \begin{bmatrix} 1 & a & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ and } SH_y = \begin{bmatrix} 1 & 0 & 0 \\ b & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Summary

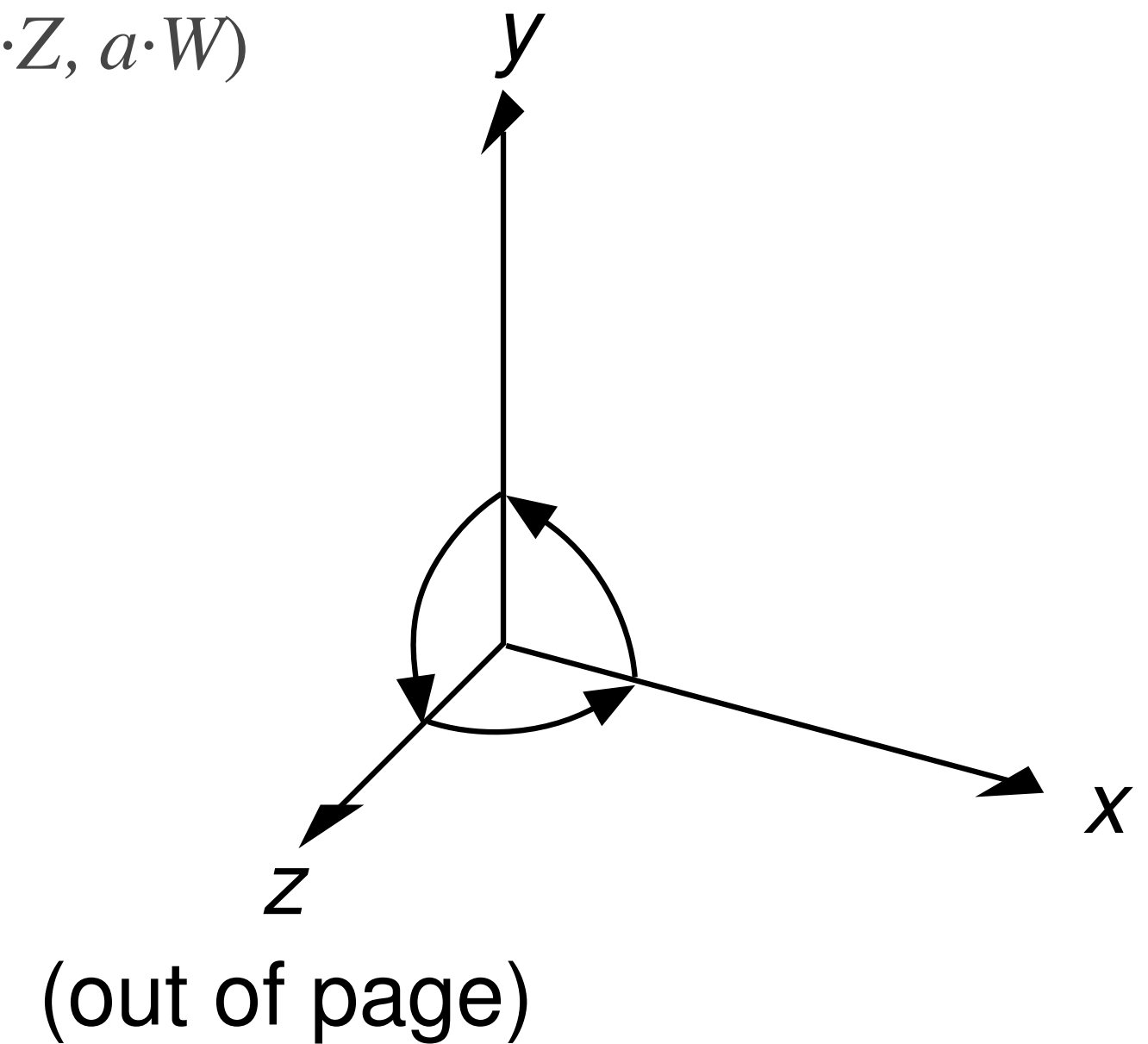
- Only one rotation, within the plane about the origin, in 2D
 - ▶ we typically assume counter-clock-wise for positive θ
- Two shear transformations, along each axis
- Further basic transformations include reflections about an axis or the origin



3D Transformations

Homogeneous 3D Transforms

- 3D transformations represented by **4x4 homogeneous matrices**
 - ▶ point (x, y, z) represented as **4-tuple** (X, Y, Z, W) with equivalence class $(a \cdot X, a \cdot Y, a \cdot Z, a \cdot W)$
 - a point with $W=0$ refer to **points at infinity / direction vector**
 - $(0, 0, 0, 0)$ being an **invalid point**
 - ▶ homogenization of (X, Y, Z, W) with $W \neq 0$ yields $(X/W, Y/W, Z/W, 1)$
- Use of a right-handed coordinate system, with **positive rotation defined counterclockwise**
 - ▶ thumb, index finger, middle finger rule



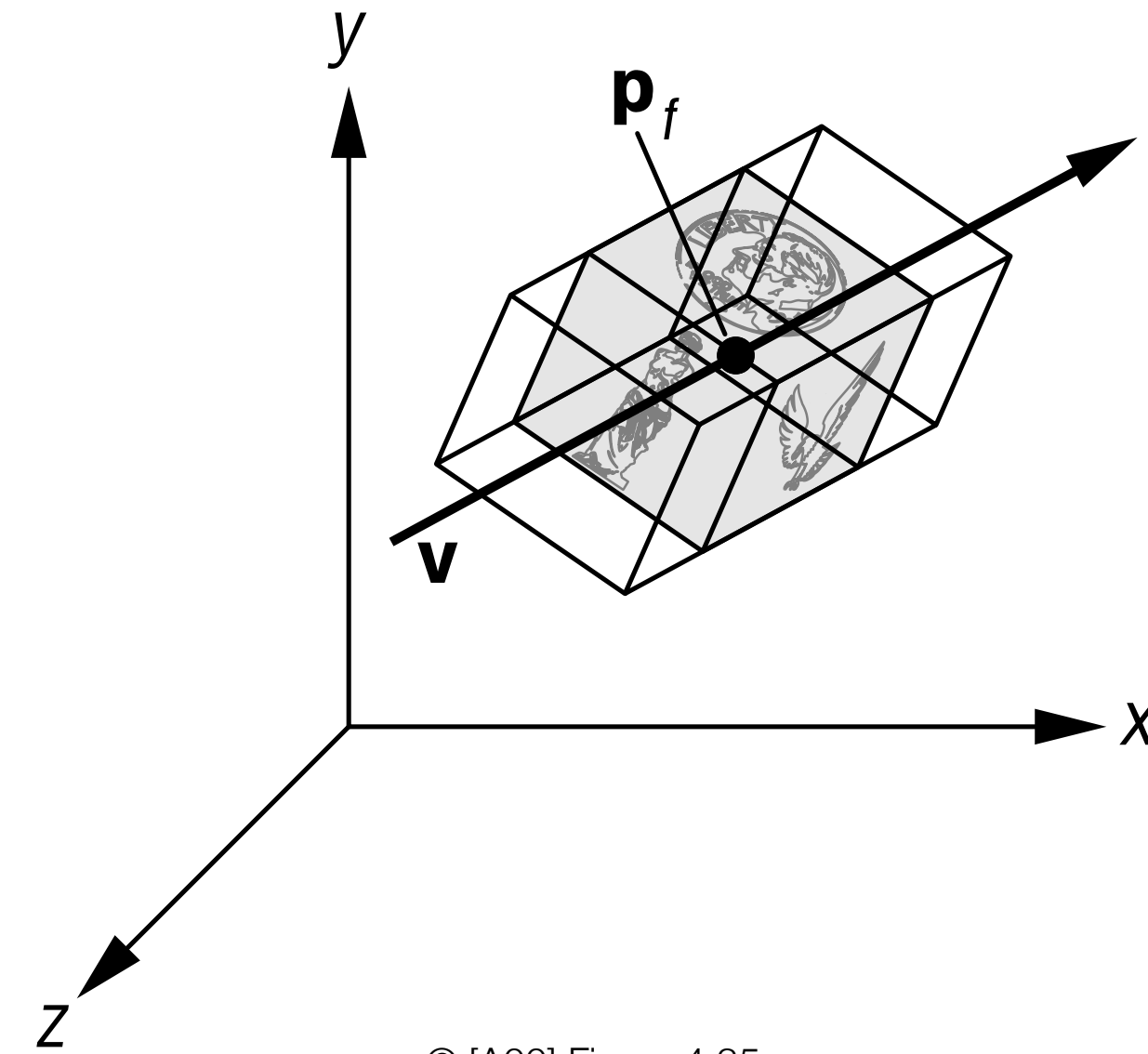
© [AW94] Figure 5.16

Translation

- Expressed as matrix multiplication $P' = T \cdot P$ with

$$T(d_x, d_y, d_z) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Applying $T(d_x, d_y, d_z)$ to $[x, y, z, 1]^T$ yields the new location $[x + d_x, y + d_y, z + d_z, 1]^T$



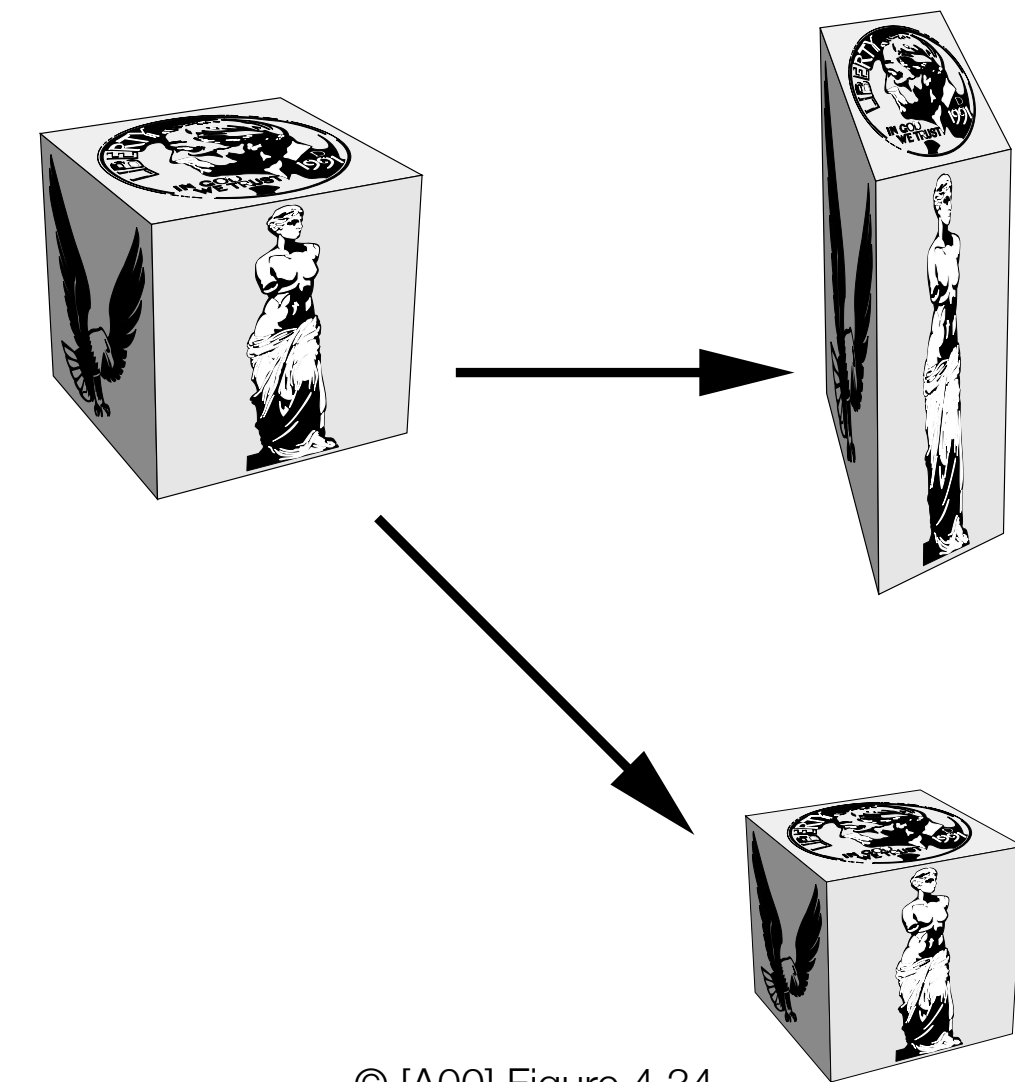
© [A00] Figure 4.35

Scaling

- Expressed as matrix multiplication $P' = S \cdot P$ with

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Applying $S(s_x, s_y, s_z)$ to $[x, y, z, 1]^T$ yields the new vector $[s_x \cdot x, s_y \cdot y, s_z \cdot z, 1]^T$



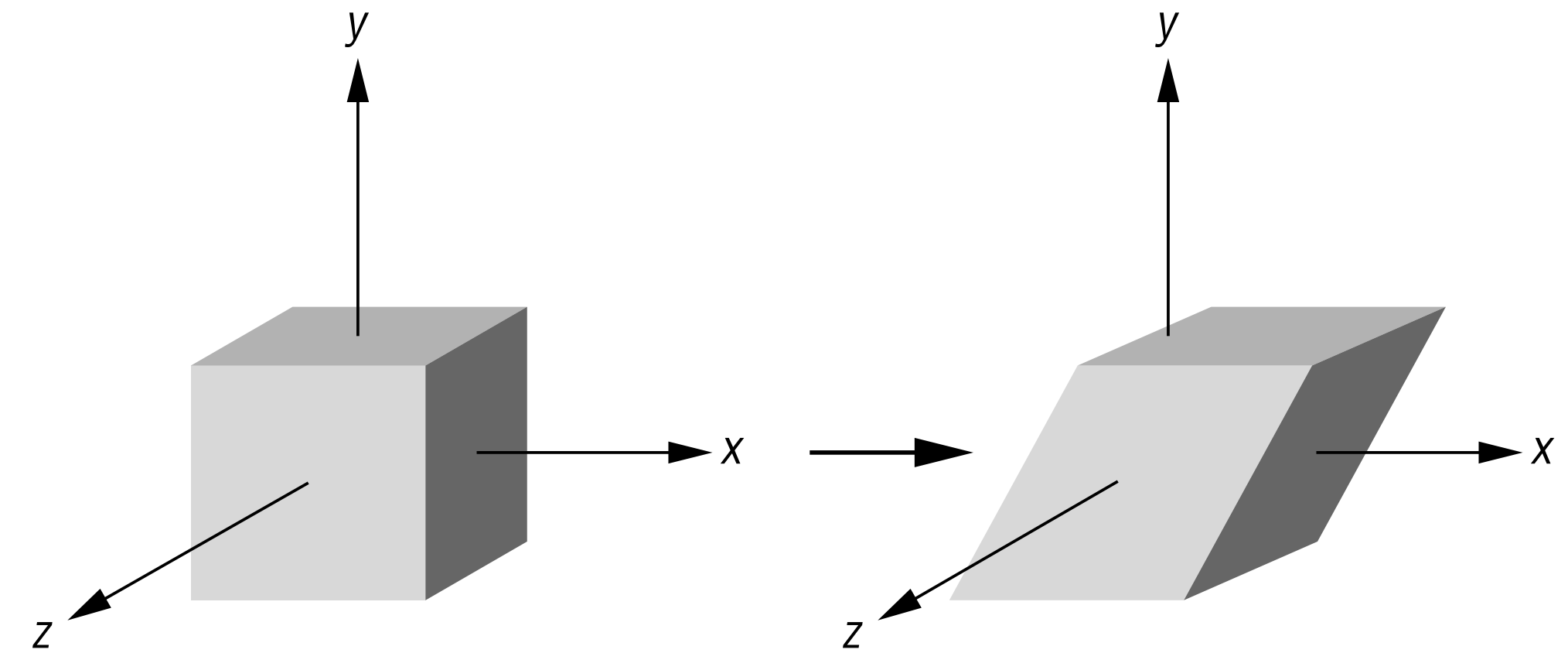
© [A00] Figure 4.34

Shear

- Expressed by three fundamental matrices
 - shear in x - y and y - z given here (parallel to the corresponding coordinate axis planes)

$$SH_{x,y}(a,b) = \begin{bmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$SH_{y,z}(b,c) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ b & 1 & 0 & 0 \\ c & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

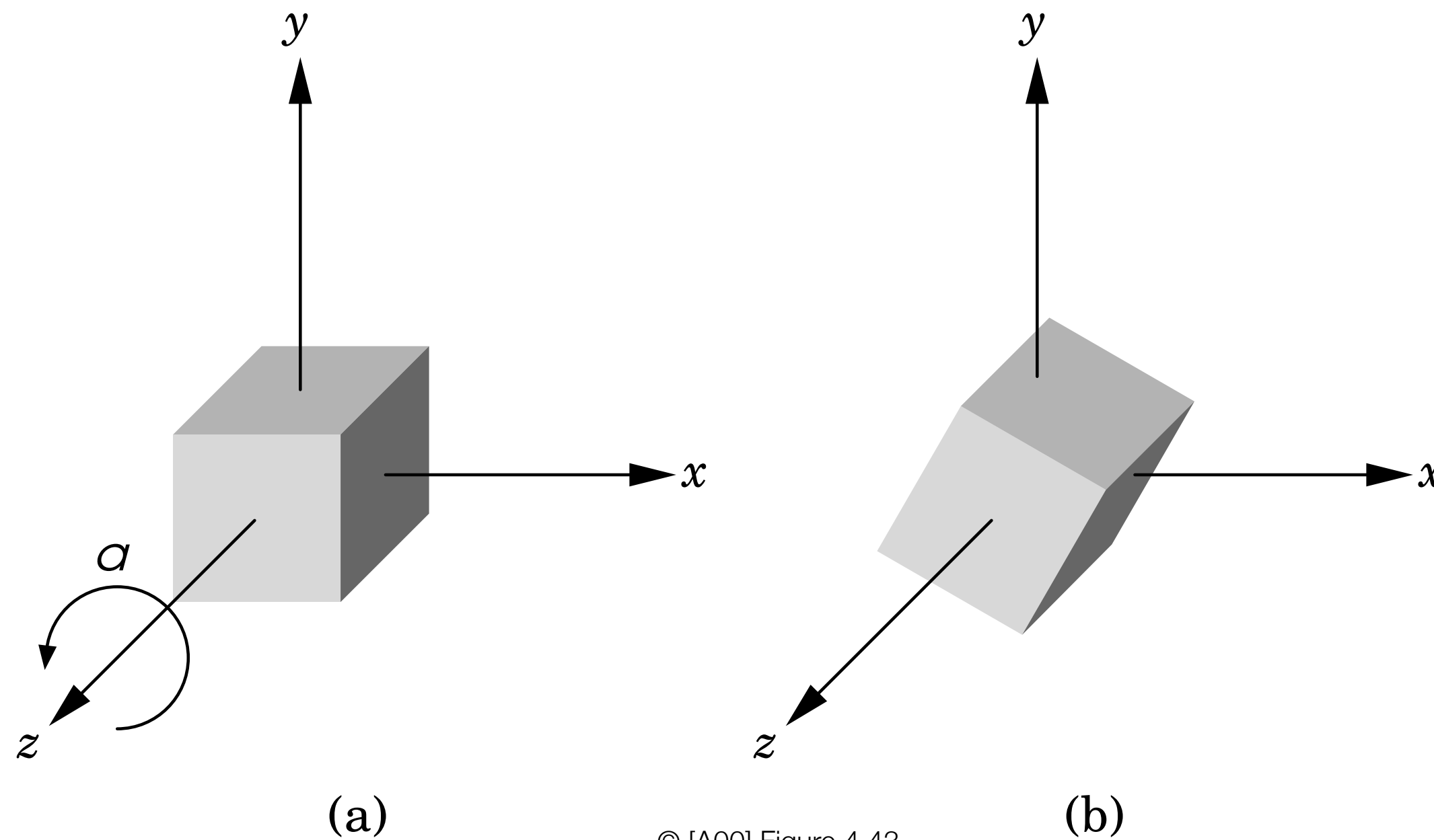


- Applying $SH_{xy}(a,b)$ to $[x, y, z, 1]^T$ thus yields $[x + a \cdot z, y + b \cdot z, z, 1]^T$
- Applying $SH_{yz}(b,c)$ to $[x, y, z, 1]^T$ thus yields $[x, y + b \cdot x, z + c \cdot x, 1]^T$

Rotation

- Rotation about the ***z*-axis** $P' = R_z \cdot P$ uses:

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



© [A00] Figure 4.43

Rotation

- Rotation about the ***z*-axis** $P' = R_z \cdot P$ uses:
- Rotation about the ***x*-axis** $P' = R_x \cdot P$ uses:
- Rotation about the ***y*-axis** $P' = R_y \cdot P$ uses:

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Copyrights

- Many figures in these slides are copyright protected as indicated. Text and all other figures are copyright protected by Renato Pajarola.
- [AS12] Electronic Book Support for Interactive Computer Graphics: A Top-Down Approach with Shader-based OpenGL by Edward Angel and Dave Shreiner. Copyright Addison Wesley 2012.
- [A00] Electronic Book Support for Interactive Computer Graphics: A Top-Down Approach Using OpenGL by Edward Angel. Copyright E. Angel 2000. (http://www.cs.unm.edu/~angel/BOOK/SECOND_EDITION/)
- [AW94] Electronic Instructors Manual (EIM) for Introduction to Computer Graphics by James Foley, Andries van Dam, Steven Feiner, John Hughes, and Richard Phillips. Copyright Addison Wesley 1994.
- [AC97] Learning Rendering CD accompanying 3D Computer Graphics by Alan Watt. Copyright A. Watt and L. Cooper 1997. (Permission for copying and distribution without direct commercial advantage. To copy otherwise, or to republish, requires specific permission.)
- [S02] Electronic Figures for Fundamentals of Computer Graphics by Peter Shirley. Copyright P. Shirley 2002. (<http://www.cs.utah.edu/~shirley/fcg/figures/>)